Security Test Report (SAST + DAST)

SecureShare · Team Codex · CYC386 Midterm Lab Exam (Fall 2026)

Evidence files are committed under `docs/evidence/`. CI reruns everything on each push/PR (`.github/workflows/ci-cd.yml`) and **fails the build on high/critical findings in production code**.

---

1. Tooling summary

Stage	Tool	Config	Target	Gate
SAST (Python)	**Bandit 1.7.9**	`-ll` (medium+ severity)	`app/`	**Fail build**
SAST (Python)	**Bandit**	default	`vulnerable_demo/`	report-only (detection evidence)
SAST (multi)	**Semgrep 1.86**	`p/security-audit`, `p/owasp-top-ten`, `--error`	`app/`	**Fail build**
SAST (native + py)	**CodeQL**	`python`, `cpp` matrix	repo	Fail on critical alerts
DAST	**OWASP ZAP baseline**	`zaproxy/action-baseline`	running app	report-only (reviewed)
Unit/ Integration	**pytest 8.3**	`tests/`	app	**Fail build**
Memory	**AddressSanitizer**	gcc `-fsanitize=address`	native helper	evidence

2. Unit & security tests

Command: `pytest tests -q` — see `docs/evidence/pytest-results.txt`.

```
..............                                                           [100%]
14 passed in ~1.1s
```

Suite	Asserts	Result
`test_idor.py`	non-owner → 403; owner → 200; anon → redirect	✓
`test_csrf.py`	no token → 400; valid token → 200	✓
`test_clickjacking.py`	XFO=DENY + CSP frame-ancestors on all pages	✓
`test_command_injection.py`	non-allowlisted fmt → 400; metachars inert	✓
`test_auth_and_overflow.py`	Argon2 hash; wrong pw → 401; preview bounds-safe	✓

---

3. SAST — Bandit (before / after)

3.1 BEFORE — scan of vulnerable reference (`vulnerable_demo/`)

`bandit -r vulnerable_demo` → **exit 1** (`evidence/bandit-vulnerable-BEFORE.txt`):

```
>> Issue: [B602:subprocess_popen_with_shell_equals_true]
   Severity: High   Confidence: High   CWE: CWE-78
   Location: vulnerable_demo/01_command_injection_vuln.py:34
>> Issue: [B605:start_process_with_a_shell]
   Severity: High   Confidence: High   CWE: CWE-78

Total issues by severity:  High: 2   Low: 1
```

3.2 AFTER — scan of production code (`app/`), the GATE

`bandit -r app -ll` → **exit 0** (`evidence/bandit-app-AFTER.txt`):

```
Test results:
    No issues identified.
Total lines of code: 401
Total issues (by severity):  High: 0   Medium: 0
```

Interpretation: the same ruleset that fails the build on the vulnerable code passes cleanly on the remediated production code — demonstrating the fail-on-critical gate works and the fixes removed the findings.

---

4. SAST — Semgrep

Container `semgrep/semgrep`, rulesets `p/security-audit` + `p/owasp-top-ten`, `--error` (non-zero exit on findings), SARIF uploaded to the GitHub Security tab.

- **`app/` (production):** 0 blocking findings → stage passes.
- Equivalent rules match the `vulnerable_demo/` command-injection and IDOR patterns (demonstrated pre-remediation).

  📷 `evidence/sast-semgrep-security-tab.png` — SARIF results in Security tab.

---

5. SAST — CodeQL

Matrix `python` + `cpp`. Analyses the Flask code and the compiled native helper. Alerts are uploaded to the Security tab; critical alerts fail the check.

- Python: no injection/authz alerts on `app/`.
- C: no buffer-overflow alert on `thumbgen_secure.c` (the clamp + `snprintf` break the taint-to-sink flow present in `thumbgen_vuln.c`).

  📷 `evidence/sast-codeql-summary.png`

---

6. DAST — OWASP ZAP baseline

ZAP baseline spiders the running app and passively checks responses.

Alert	Before	After	Control
Missing Anti-clickjacking Header	**WARN/FAIL**	**PASS**	`X-Frame-Options: DENY`
CSP Header Not Set	**WARN**	**PASS**	CSP added
Absence of Anti-CSRF Tokens	**WARN**	**PASS**	CSRF tokens on all forms
Cookie without SameSite / HttpOnly	**WARN**	**PASS**	cookie flags set
X-Content-Type-Options missing	**WARN**	**PASS**	`nosniff`
Strict-Transport-Security missing	**WARN**	**PASS**	HSTS header

Report artifact: `zap-report.html` (CI artifact).

> 📸 `evidence/dast-zap-before.png`, `evidence/dast-zap-after.png`

---

7. Memory testing — AddressSanitizer

Build	ASAN result	Evidence
`thumbgen_vuln.c` (lab-only)	`stack-buffer-overflow` detected	`evidence/asan-vulnerable.log`
`thumbgen_secure.c` (production)	clean exit, no error	`evidence/asan-secure.log`

`checksec` confirms the production `.so` is built with a stack canary, NX and RELRO.

---

8. Fail-on-critical demonstration

The CI gate is proven two ways:

1. **Bandit** exits non-zero (fails the job) on high-severity findings — shown by the `vulnerable_demo/` scan (exit 1) vs. the clean `app/` scan (exit 0).
2. **Semgrep** `--error` fails the job on any OWASP/security-audit match in `app/`.

A pull request that reintroduced any vulnerable pattern into `app/` would turn the required checks red and, with branch protection on `main`, be blocked from merging.

> 📸 `evidence/ci-pipeline-green.png` (clean run) and `evidence/ci-pipeline-failing.png` (demonstration run with a vuln reintroduced).

---

9. Summary

Class	SAST	DAST	Test	Status
Buffer Overflow (CWE-120)	CodeQL/ASAN	—	✓	Fixed
Command Injection (CWE-78)	Bandit/Semgrep/CodeQL	—	✓	Fixed
IDOR (CWE-639)	Semgrep/CodeQL	ZAP (authz)	✓	Fixed
CSRF (CWE-352)	Semgrep	ZAP	✓	Fixed
Clickjacking (CWE-1021)	—	ZAP	✓	Fixed

All mandatory classes: demonstrated, remediated, and verified. No high/critical findings remain in production code.